Introduction – Présentation du Projet

Objectif

Ce projet vise à développer un outil de reporting pour identifier et gérer les fichiers dense.

Fait par :

LECAS Clément

AMORY Ryan

Made with Gamma

Sommaire

1 Fonctionnalités

2 Exemple d'Utilisation

3 Programmes utilisés

4 Explication des programmes

5 Problèmes rencontrés

6 Solutions trouvées

7 Explication détaillée des codes → *hors exposition*

Made with Gamma

1. Fonctionnalités du Logiciel

Analyse de Répertoire

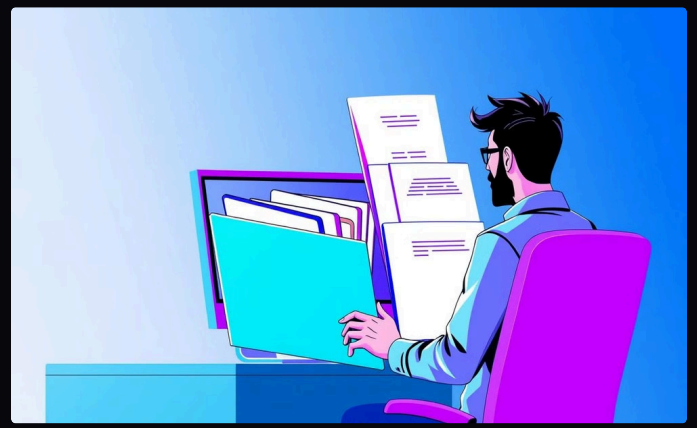
Le logiciel analyse automatiquement un répertoire spécifié par l'utilisateur, en identifiant tous les fichiers qu'il contient.

Suppression de fichiers

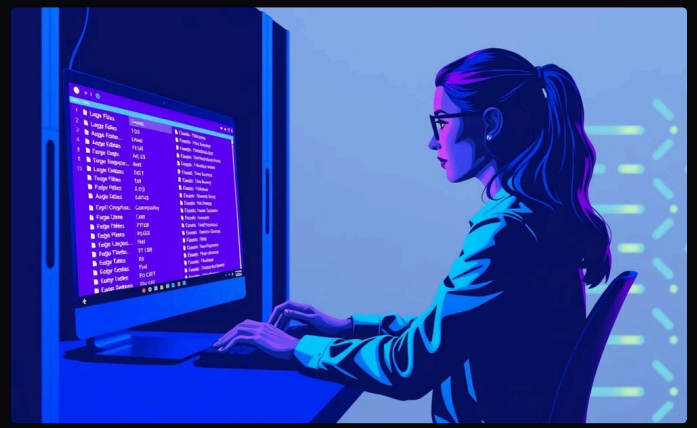
Une fois l'analyse terminée, le logiciel trie tous les fichiers du répertoire par taille, du plus grand au plus petit et permet à l'utilisateur de supprimer certains fichiers.

Made with Gamma

2. Exemple d'Utilisation



Gestion de Fichiers Personnels



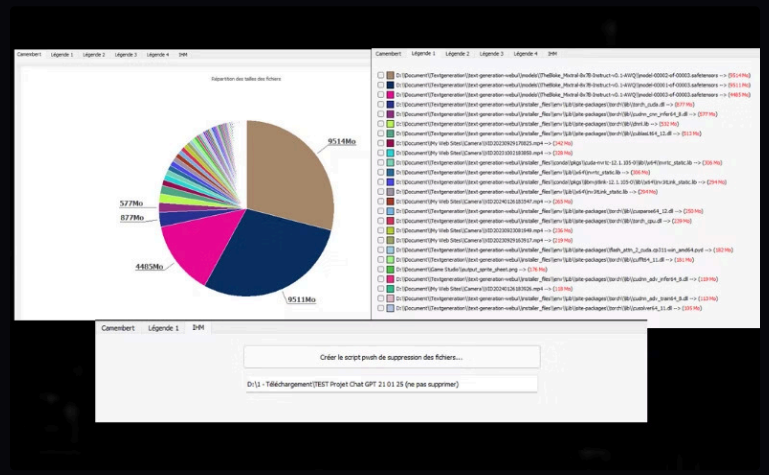
Administration Système



Analyse des Données

Made with Gamma

3. Programmes Utilisés





Programmes graphique :

- Creation_Boutons.py
- Creation_Camembert.py
- Creation_Legendes.py
- Creation_Onglets.py

Programmes de fonctionnement :

- Run_Reporting_Tool.ps1
- Script_1.py
- Script_2.py
- Script_3.py

Made with Gamma

4. Explication des Programmes

Creation_Boutons.py

Programme permettant la création de boutons.

Crédit: Olivier ECKLE

Creation_Camembert.py

Programme permettant la création d'un graphique de type camembert.

Crédit: Olivier ECKLE

Creation_Legendes.py

Programme permettant la création des cases de suppression de fichiers.

Crédit: Olivier ECKLE

Creation_Onglets.py

Programme permettant la création d'onglets interactive.

Crédit: Olivier ECKLE

Run_Reporting_Tool.ps1

Fait la liaison entre chaque scripts de l'outil.

Script_1.py

Choix des données.

Script_2.py

Explore, tri, filtre et export ces données.

Script_3.py

Interface graphique.

Made with Gamma

5. Problèmes Rencontrés

Nous avons rencontrer quelque problèmes durant la création de l'outil

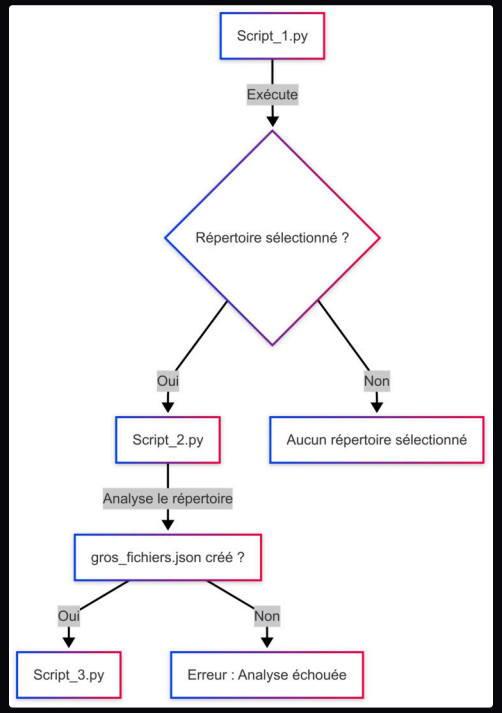
1. Liaison Powershell
2. Problème de connaissance
3. Création du script de supression



Made with Gamma

6. Solutions Trouvées

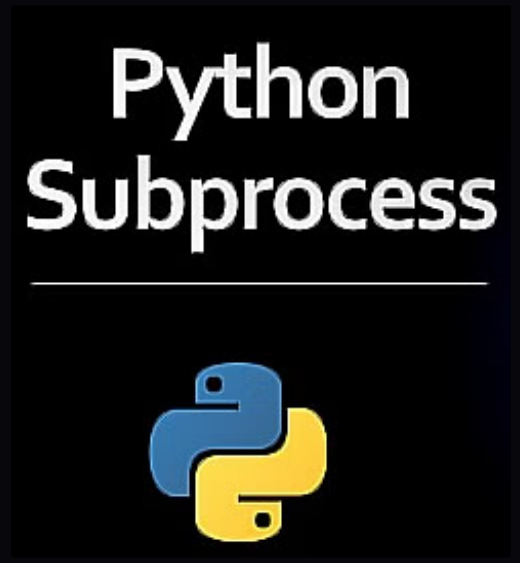
Utilisation de la sortie du Script_1.py



Recherche intensive



Utilisation du module subprocesses



Made with Gamma

Conclusion

Durant ce projet, nous avons pu développer nos compétences techniques et humaines. Tant sur la programmation que sur l'organisation.

Clément aura plus développé sa connaissance sur le langage python, dû à ses Lacunes, alors que Ryan aura plus développé son côté organisationnel dû à ses connaissances solides en Python.

Cela a été un projet intéressant et concret, qui nous a immergé dans une situation d'entreprise.



Merci pour votre attention !

Made with Gamma

7. Explication des codes



Made with Gamma

Creation_Boutons.py

Programme :

from PyQt5.QtWidgets import QWidget, QVBoxLayout, QPushButton, QLineEdit from PyQt5.QtCore import Qt

class Boutons: """ Création d'un objet graphique contenant un bouton et une textBox. Le chemin du répertoire de base est passé dans le 1er argument. Il est affiché dans la textBox. Le nom de la 'callcack' associé au 'clic' sur le bouton est passé en 2ème argument. """

```
def __init__(self, repertoire_base, callback):
    self.repertoire_base = repertoire_base
    self.callback = callback

def dessine_boutons(self):
    """
        Retourne une Widget Layout PyQt contenant un bouton et une textBox.
    """

    # Création de l'objet graphique et son layout contenant un bouton et une textbox
    boutons = QWidget()
    zone_boutons = QVBoxLayout(boutons)
    zone_boutons.setSpacing(10)
    zone_boutons.setAlignment(Qt.AlignTop)

    # Création du Bouton
    bouton_genere_delete_script = QPushButton("Créer le script pwsh de suppression des fichiers...")
    bouton_genere_delete_script.setFixedSize(500, 40)
    bouton_genere_delete_script.clicked.connect(self.callback)
    zone_boutons.addWidget(bouton_genere_delete_script, alignment=Qt.AlignHCenter)

    # Création de la textBox
    text_repertoire_base = QLineEdit()
    text_repertoire_base.setText(self.repertoire_base)
    text_repertoire_base.setFixedSize(490, 30)
    zone_boutons.addWidget(text_repertoire_base, alignment=Qt.AlignHCenter)

    # Retour de l'objet graphique et son layout contenant un bouton et une textBox
    return boutons
```

Made with Gamma

Creation_Boutons.py

Explication 1

Importations

- `QWidget` : Représente un widget PyQt de base.
- `QVBoxLayout` : Définit un layout vertical pour organiser les widgets.
- `QPushButton` : Représente un bouton cliquable.
- `QLineEdit` : Champ de texte permettant l'entrée d'un utilisateur.
- `Qt` : Contient des constantes utilisées pour l'alignement et d'autres options d'affichage.

Définition de la classe `Boutons`

- Cette classe représente un composant graphique avec un **bouton** et une **zone de texte**.
- `repertoire_base` : Stocke un chemin qui sera affiché dans la zone de texte.
- `callback` : Fonction exécutée lorsqu'on clique sur le bouton.

Constructeur de la classe

- Initialise les attributs `repertoire_base` et `callback` à partir des paramètres passés.

Méthode `dessine_boutons`

- Cette méthode **crée et retourne** un widget contenant le bouton et la zone de texte.

Création du widget principal et de son layout

- `boutons = QWidget()` : Création d'un conteneur principal.
- `zone_boutons = QVBoxLayout(boutons)` : Layout vertical pour organiser les éléments.
- `setSpacing(10)` : Définit un espacement de 10 pixels entre les éléments.
- `setAlignment(Qt.AlignTop)` : Aligne les éléments en haut du conteneur.

Création du bouton

- `QPushButton(...)` : Crée un bouton avec un texte explicite.
- `setFixedSize(500, 40)` : Définit une taille fixe (500px de large, 40px de haut).
- `clicked.connect(self.callback)` : Relie le clic du bouton à la fonction `callback`.
- `addWidget(..., alignment=Qt.AlignHCenter)` : Ajoute le bouton dans le layout, centré horizontalement.

Made with Gamma

Creation_Boutons.py

Explication 2

Création de la zone de texte

- `QPushButton(...)` : Crée un bouton avec un texte explicite.
- `setFixedSize(500, 40)` : Définit une taille fixe (500px de large, 40px de haut).
- `clicked.connect(self.callback)` : Relie le clic du bouton à la fonction `callback`.
- `addWidget(..., alignment=Qt.AlignHCenter)` : Ajoute le bouton dans le layout, centré horizontalement.

Création de la zone de texte

- `QLineEdit()` : Crée une zone de texte.
- `setText(self.repertoire_base)` : Remplit la zone avec le chemin du répertoire de base.
- `setFixedSize(490, 30)` : Fixe une taille de 490x30 pixels.
- `addWidget(..., alignment=Qt.AlignHCenter)` : Ajoute la zone de texte dans le layout, centrée horizontalement.

Retour du widget contenant le bouton et la zone de texte

- Renvoie le widget complet avec son contenu.

Résumé 💡

Ce code définit une classe `Boutons` qui génère un **bouton** et une **zone de texte** dans un **layout vertical**.

- **Le bouton** exécute une fonction `callback` au clic.
- **La zone de texte** affiche un chemin (`repertoire_base`).
- La méthode `dessine_boutons()` retourne le widget complet.

Made with Gamma

Creation_Camembert.py

Programme :

from PyQt5.QtChart import QChart, QChartView, QPieSeries from PyQt5.QtGui import QFont

class Camembert: """ Création d'un objet graphique contenant une réprésentation statistique sous forme de camembert. La série de données représente la liste passée dans le 1er argument. Les couleurs des tranches du camembert sont celle de la liste passée en 2ème argument. """ def init(self, liste_fichiers, liste_couleurs): self.liste_fichiers = liste_fichiers self.liste_couleurs = liste_couleurs

```
def dessine_camembert(self):
    """
    Retourne une Widget Layout PyQt contenant un graphique circulaire type camembert.
    """
    if not self.liste_fichiers:
        raise ValueError(f"La liste doit contenir au moins 1 fichier.")

    # Création de la série de données pour le graphique
    series = QPieSeries()
    series.setLabelsVisible(True)
    taille_totale = 0
    for liste in self.liste_fichiers:
        taille_totale += liste[1]
```

```
# Création des différentes tranches du camembert
font = QFont("Arial Narrow", 12, QFont.Bold)
for path_fichier, taille_fichier in self.liste_fichiers:
    etiquette = f"{taille_fichier // 1048576}Mo"
    pourcentage = taille_fichier / taille_totale * 100
    slice_ = series.append(etiquette, pourcentage)
    slice_.setBrush(self.liste_couleurs[len(series)-1])  # Couleurs dynamiques
    slice_.setLabelFont(font)
    slice_.setLabelPosition(slice_.LabelPosition.LabelOutside)

# Affichage d'une étiquette pour les grandes tranches du camembert
for slice_ in series.slices():
    slice_.setLabelVisible(slice_.angleSpan() > 6)

# Création du camembert complet
fromage = QChart()
fromage.addSeries(series)
fromage.setTitle("Répartition des tailles des fichiers")
fromage.legend().hide()

# Configuration du graphique avec QChartView
layout_fromage = QChartView(fromage)

# Retour de l'objet graphique et son layout contenant le camembert
return layout_fromage
```

Made with Gamma

Creation_Camembert.py

Programme :

Importations

- `QChart` : Conteneur principal du graphique.
- `QChartView` : Widget pour afficher le graphique.
- `QPieSeries` : Permet de créer un diagramme circulaire.
- `QFont` : Permet de personnaliser la police des étiquettes.

Définition de la classe `Camembert`

- Cette classe permet de **générer un graphique circulaire (camembert)** basé sur une liste de fichiers et leurs tailles.

Constructeur

- `liste_fichiers` : Liste contenant des tuples (`chemin_fichier`, `taille_fichier`).
- `liste_couleurs` : Liste de couleurs associées aux différentes portions du camembert.

Méthode `dessine_camembert()`

- Cette méthode **crée et retourne** un widget contenant le graphique circulaire.

Vérification de la liste des fichiers

- Empêche de créer un camembert vide.

Création de la série de données

- `series` représente les différentes tranches du camembert.

Calcul de la taille totale des fichiers

- Additionne toutes les tailles pour **calculer les pourcentages**.

Ajout des tranches dans le camembert

- **Transforme les tailles en Mo**.
- **Ajoute une tranche** avec la taille et la couleur associée.

Affichage des étiquettes uniquement pour les grandes tranches

- N'affiche pas les étiquettes si la tranche est trop petite.

Création et configuration du graphique `fromage = QChart()`

- **Ajoute la série au graphique**.
- **Cache la légende** (les étiquettes suffisent).

Retour du widget graphique

- Transforme le graphique en **widget affichable dans PyQt**.

Résumé 💡

- Génère un graphique camambert des données.

Made with Gamma

Creation_Legendes.py

Programme :

from PyQt5.QtWidgets import QWidget, QVBoxLayout, QLabel, QHBoxLayout, QCheckBox
from PyQt5.QtCore import Qt

class Legendes: """ Création d'un objet graphique contenant une série de lignes de légendes. La série de données représente la liste passée dans le 1er argument. Les couleurs des tranches du camembert sont celle de la liste passée en 2ème argument. Le nombre de lignes de légendes est passé dans le 4ème argument. Le numéro de la 1ère ligne de légendes pris dans la liste des données est passé dans le 3ème argument... """ def init(self, liste_fichiers, liste_couleurs, num_legende_start, nb_legende_par_page): self.liste_fichiers = liste_fichiers self.liste_couleurs = liste_couleurs self.num_legende_start = num_legende_start self.nb_legende_par_page = nb_legende_par_page self.num_legende_stop = min(len(liste_fichiers), num_legende_start+self.nb_legende_par_page) self.cases_a_cocher = []

```
def dessine_legendes(self):
    """
    Retourne une Widget Layout PyQt contenant une liste de légendes.
    """

    # Création de l'objet graphique et son layout contenant un certain nombre de lignes de légendes
    legendes = QWidget()
    zone_legendes = QVBoxLayout(legendes)
    zone_legendes.setContentsMargins(0, 20, 0, 20)
    zone_legendes.setAlignment(Qt.AlignTop)
```

```
    # Ajouter des datas à chaque légende
    for num_slice in range(self.num_legende_start, self.num_legende_stop):
        # Création de la case à cocher
        case_a_cocher = QCheckBox()
        case_a_cocher.setChecked(False)  # Définir l'état initial de la case (cochée ou non)
        self.cases_a_cocher.append(case_a_cocher)

        # Création d'un petit carré coloré
        rectangle_colore = QWidget()
        rectangle_colore.setFixedSize(16, 16)  # Taille du carré
        couleur = self.liste_couleurs[num_slice].name()
        rectangle_colore.setStyleSheet(f"background-color: {couleur}; border: 1px solid black;")

        # Composition du contenu textuel de la légende
        chemin_fichier = self.liste_fichiers[num_slice][0]  # Extrait le chemin et nom du fichier
        taille_fichier = self.liste_fichiers[num_slice][1] // 1048576  # Extrait la taille du fichier
        etiquette_legende = f"<span style='color:black;font-family:Arial Narrow'>{chemin_fichier} --> (</span><span style='color:red;'>{taille_fichier} Mo</span>)"
        texte_legende = QLabel(etiquette_legende)
```

Made with Gamma

Creation_Legendes.py

Programme :

```
        # Assemblage des éléments d'une ligne de légende
        ligne_legende = QHBoxLayout()
        ligne_legende.setAlignment(Qt.AlignLeft)
        ligne_legende.setContentsMargins(5, 0, 5, 5)
        ligne_legende.stretch(2)
        ligne_legende.addWidget(case_a_cocher)
        ligne_legende.addWidget(rectangle_colore)
        ligne_legende.addWidget(texte_legende)

        # Création d'une ligne de légende
        legende = QWidget()
        legende.setLayout(ligne_legende)
        legende.setContentsMargins(0, 0, 0, 0)

        zone_legendes.addWidget(legende)
        zone_legendes.setSpacing(0)

    # Retour de l'objet graphique et son layout contenant 25 lignes de légendes
    return legendes

def recupere_etats_cases_a_cocher(self):
    """
    Retourne une liste de boolean indiquant, pour cette page de légendes (cet objet), les états des cases à cocher...
    """
```

```
etats = []
for case in self.cases_a_cocher:
    etats.append(case.isChecked())
return etats
```

Made with Gamma

Creation_Legendes.py

Programme :

Importations

- `QWidget` : Conteneur pour chaque légende.
- `QVBoxLayout` : Layout vertical pour organiser les légendes.
- `QHBoxLayout` : Layout horizontal pour chaque ligne de légende.
- `QLabel` : Texte pour afficher le nom du fichier.
- `QCheckBox` : Cases à cocher pour activer/désactiver des fichiers.
- `Qt` : Pour les options d'alignement.

Définition de la classe `Legendes`

- Cette classe permet de **générer une liste de légendes associées aux tranches du camembert**.

Constructeur

- `liste_fichiers` : Liste des fichiers et leurs tailles.
- `liste_couleurs` : Couleurs associées aux fichiers.
- `num_legende_start` : Indice de la première légende à afficher.
- `nb_legende_par_page` : Nombre de légendes affichées par page.

Calcul du dernier indice

- **Évite de dépasser la taille de la liste**.

Méthode `dessine_legendes()`

- Cette méthode **crée et retourne un widget contenant les légendes**.

Création du conteneur principal

- Crée un **layout vertical**.

Boucle de création des légendes

- **Itère sur les fichiers** à afficher.

Ajout des éléments graphiques

- **Ajoute une case à cocher** pour chaque fichier.
- **Ajoute un carré coloré** correspondant au camembert.
- Crée un texte affichant le nom du fichier et sa taille.

Assemblage des éléments

- **Organise chaque légende sur une ligne**.

Ajout dans le layout global

- Ajoute chaque légende au layout principal.

Méthode `recupere_etats_cases_a_cocher()`

- **Récupère l'état de chaque case à cocher** (activée ou non).

Résumé 💡

- Génère une liste de légendes associées avec cases à cocher.

Made with Gamma

Creation_Onglets.py

Programme :

from PyQt5.QtWidgets import QWidget, QTabWidget, QVBoxLayout, QMainWindow

class Onglets(QMainWindow): """ Création Une fenêtre graphique contenant un conteneur d'onglets. Sans argument ! """ def init(self): super().init()

```
    self.setWindowTitle("Résultat de recherche des 'gros' fichiers...")
    self.setGeometry(200, 100, 1000, 700)

    # Créer un QTabWidget
    self.onglets = QTabWidget()
    self.setCentralWidget(self.onglets)


def add_onglet(self, titre, layout_a_ajouter):
    """
    Ajout d'un onglet contenant le layout passé en second argument à la méthode.
    Le 1er argument est le titre de l'onglet
    """
    # création d'un onglet...
    onglet = QWidget()
    layout_onglet = QVBoxLayout()
    layout_onglet.addWidget(layout_a_ajouter)
    onglet.setLayout(layout_onglet)

    # Ajout de l'onglet au QTabWidget
    self.onglets.addTab(onglet, titre)
```

Explication 1 :

Importation

- QWidget : Élément de base pour tous les composants graphiques.
- QTabWidget : Widget permettant de gérer plusieurs onglets.
- QVBoxLayout : Gestionnaire de mise en page verticale.
- QMainWindow : Fenêtre principale contenant les éléments de l'interface.

Définition de la classe Onglets

- Définit une classe Onglets qui hérite de QMainWindow, la fenêtre principale de PyQt5.

Docstring de la classe

- Explique que cette classe crée une fenêtre avec des onglets et ne nécessite aucun paramètre.

Constructeur de la classe

- __init__() : Constructeur de la classe, appelé lors de l'instanciation.
- super().__init__() : Initialise QMainWindow.

Définition du titre et de la taille de la fenêtre

- setWindowTitle(...) : Définit le titre de la fenêtre.
- setGeometry(x, y, largeur, hauteur) : Définit la position (200,100) et la taille (1000x700).

Made with Gamma

Creation_Onglets.py

Explication 2 :

Création du widget d'onglets

- `QTabWidget()` : Crée un conteneur d'onglets.
- `setCentralWidget(self.onglets)` : Place ce conteneur au centre de la fenêtre.

Méthode pour ajouter des onglets

- Définit une méthode `add_onglet()` prenant :
  - `titre` : Nom de l’ onglet.
  - `layout_a_ajouter` : Contenu à afficher dans l'onglet.

Création de l'onglet et ajout du layout

- `QWidget()` : Crée un nouvel onglet.
- `QVBoxLayout()` : Crée un layout vertical.
- `layout_onglet.addWidget(layout_a_ajouter)` : Ajoute le widget fourni en argument au layout.
- `onglet.setLayout(layout_onglet)` : Associe le layout à l'onglet.

Ajout de l'onglet au conteneur

- `addTab(onglet, titre)` : Ajoute l’ onglet à `QTabWidget` avec un titre donné.

Résumé 💡

- **Ce script permet donc de créer une interface avec des onglets dynamiquement.**

Made with Gamma

Run_Reporting_Tools.ps1

Programme :

```
$OutputEncoding = [System.Text.Encoding]::UTF16
$rep_base = (python .\Script_1.py)
if (-not $rep_base) {
    Write-Host "Aucun répertoire sélectionné."
    exit
}
Write-Host "Directory passed to Script_2: $rep_base"
python .\Script_2.py $rep_base

if (-not (Test-Path "gros_fichiers.json")) {
  Write-Host "Erreur lors de l'analyse du répertoire."
  exit
}
python .\Script_3.py
```

Explication 1 :

Définition de l'encodage

- Définit l'encodage en **UTF-16** pour s'assurer que les caractères spéciaux (ex: accents, espaces) sont correctement affichés.

Exécution de Script_1.py

- **Lance** Script_1.py qui ouvre une boîte de dialogue pour sélectionner un répertoire.
- La sortie de Script_1.py (chemin du dossier sélectionné) est stockée dans la variable $rep_base.

Vérification si un répertoire a été sélectionné

- Si $rep_base est vide (-not $rep_base), cela signifie qu'aucun répertoire n'a été sélectionné.
- Affiche un message et **quitte le script** (exit).

Exécution de Script_2.py

- **Affiche le répertoire sélectionné**.
- **Exécute** Script_2.py en lui passant le répertoire sélectionné comme argument.
- Script_2.py va analyser ce répertoire et générer un fichier gros_fichiers.json avec les fichiers volumineux trouvés.

Made with Gamma

Run_Reporting_Tools.ps1

Explication 2 :

Vérification de la création du fichier gros_fichiers.json

- Vérifie si le fichier gros_fichiers.json a bien été créé par Script_2.py (Test-Path vérifie l'existence d'un fichier).
- **Si le fichier n'existe pas**, affiche un message d'erreur et **quitte le script**.

Exécution de Script_3.py

Lance Script_3.py qui va :

1. Charger les données du fichier gros_fichiers.json.
2. Afficher un graphique **camembert** et des **légendes** des fichiers volumineux.
3. Proposer une interface pour **supprimer certains fichiers via un script PowerShell**.

Résumé 💡

- **Demande à l'utilisateur** de sélectionner un dossier (Script_1.py).
- **Analyse le dossier** pour identifier les fichiers volumineux (Script_2.py).
- **Vérifie si le fichier** gros_fichiers.json **est bien généré**.
- **Affiche une interface graphique** pour visualiser et gérer les fichiers (Script_3.py).

Made with Gamma

Script_1.py

Programme :

##Partie 1:

import sys from pathlib import Path from PyQt5.QtWidgets import QApplication, QFileDialog, QWidget

##Partie 2:

def choisir_repertoire() -> Path | None: """ Ouvre une boîte de dialogue pour sélectionner un répertoire Nous donne le chemin sélectionné sous forme d'objet "Path", ou None si annuléer """ app = QApplication(sys.argv) widget = QWidget() widget.hide() # Empêche l'affichage d'une fenêtre superflue

```
    dossier = QFileDialog.getExistingDirectory(widget, "Sélectionnez un répertoire", str(Path.home()))

    return Path(dossier) if dossier else None
```

##Partie 3:

def main(): # Sélection du répertoire repertoire_de_base = choisir_repertoire()

```
# Validation et affichage
if repertoire_de_base:
    print(f"Répertoire sélectionné : {repertoire_de_base}")
else:
    print("Aucun répertoire sélectionné.")

sys.exit()
```

if name == 'main': main()

Made with Gamma

Script_1.py

Explication :

Importation

- `sys` : Gère les arguments de la ligne de commande.
- `Path` : Manipule les chemins de fichiers.
- `QApplication` : Nécessaire pour exécuter une application PyQt5.
- `QFileDialog` : Permet d’ ouvrir une boîte de dialogue pour sélectionner un répertoire.
- `QWidget` : Classe de base pour l'interface graphique.

Fonction de sélection de répertoire

- Renvoie un objet `Path` si un dossier est sélectionné, `None` sinon.
- Initialise une application PyQt5.
- Crée un widget invisible pour éviter une fenêtre superflue.
- `QFileDialog.getExistingDirectory()` : Ouvre une boîte de dialogue pour choisir un dossier.
- `str(Path.home())` : Définit le dossier de départ (dossier utilisateur).
- Retourne le chemin sélectionné sous forme d’ objet `Path`, sinon `None`.

Fonction principale

- Point d'entrée du programme.
- Stocke le chemin du dossier sélectionné.
- Affiche le chemin choisi ou un message d'erreur.
- Ferme l"application proprement.

Lancement du script

- Exécute `main()` uniquement si le script est lancé directement.

Résumé 💡

- Ce script ouvre une boîte de dialogue pour sélectionner un dossier et affiche le chemin sélectionné.

Made with Gamma

Script_2.py

Programme :

##Partie numéro 1:

import json import sys from pathlib import Path

##Partie numéro 2:

def build_list(repertoire_de_base: Path) -> list: """ Explore récursivement le répertoire pour récupérer la liste des fichiers avec leur taille. Chaque élément est une liste : [chemin absolu (str), taille en octets (int)]. """ liste_fichiers = [] for fichier in repertoire_de_base.rglob('*'): if fichier.is_file(): try: taille = fichier.stat().st_size except Exception: taille = 0 liste_fichiers.append([str(fichier.resolve()), taille]) if liste_fichiers: print("Some files detected:", liste_fichiers[:3]) return liste_fichiers

##Partie numéro 3:

def sort_function(liste_fichiers: list) -> list: """Trie les fichiers du plus gros au plus petit.""" return sorted(liste_fichiers, key=lambda x: x[1], reverse=True)

##Partie numéro 4:

def filter_function(liste_fichiers: list, TAILLE_MINI_FICHIER_EN_MEGA_OCTET: float, NB_MAXI_FICHIERS: int) -> list: """ Filtre les fichiers selon une taille minimale et un nombre maximal. """ taille_min_octets = TAILLE_MINI_FICHIER_EN_MEGA_OCTET * 1048576 # Conversion Mo → octets fichiers_filtres = [f for f in liste_fichiers if f[1] >= taille_min_octets]

Garde seulement les fichiers assez grands return fichiers_filtres[:NB_MAXI_FICHIERS] # Limite à NB_MAXI_FICHIERS fichiers

##Partie numéro 5:

def build_json(liste_fichiers: list, nom_fichier: str) -> None: """ Sauvegarde la liste des fichiers sous forme de JSON. """ fichiers_json = [[chemin.replace('\', '\\'), taille] for chemin, taille in liste_fichiers]

```
with open(nom_fichier, "w", encoding="utf-8") as f:
    json.dump(fichiers_json, f, indent=4)
print(f"Fichier JSON '{nom_fichier}' généré avec succès.")
```

##Partie numéro 6:

def main(): # Initialisation : Si un argument est passé, on l'utilise sinon le répertoire courant repertoire_de_base = Path(sys.argv[1]) if len(sys.argv) > 1 else Path.cwd()

```
print(f"Analyse du répertoire : {repertoire_de_base}")

# Exploration
liste_fichiers = build_list(repertoire_de_base)
print(f"{len(liste_fichiers)} fichiers détectés.")
```

Made with Gamma

Script_2.py

Programme :

```
# Tri
fichiers_tries = sort_function(liste_fichiers)

# Filtrage
TAILLE_MIN = 0  # Mo
NB_MAX = 100
fichiers_final = filter_function(fichiers_tries, TAILLE_MIN, NB_MAX)
print(f"{len(fichiers_final)} fichiers retenus après filtrage.")

# Export
build_json(fichiers_final, "gros_fichiers.json")
```

`if name == 'main': main()`

Explication 1 :

Importation

- `json` : Permet d'enregistrer et lire les fichiers en format JSON.
- `sys` : Gère les arguments de la ligne de commande.
- `Path` : Facilite la manipulation des chemins de fichiers.

Fonction `build_list()` (Exploration du répertoire)

- Prend un chemin de répertoire (`Path`).
- Retourne une liste contenant les fichiers et leur taille.
- Initialise une liste pour stocker les fichiers trouvés.
- `rglob('*')` : Parcourt **récursivement** tous les fichiers du dossier.
- Vérifie si l'élément trouvé est un fichier (et non un dossier).
- `fichier.stat().st_size` récupère la taille du fichier en octets.
- En cas d'erreur d'accès au fichier, on met une taille de `0` octets.
- Ajoute `[chemin absolu, taille]` à `liste_fichiers`.
- Affiche les **3 premiers fichiers détectés** (pour le debug).
- Retourne la liste complète des fichiers trouvés.

Fonction `sort_function()` (Tri des fichiers)

- Trie la liste des fichiers du **plus gros au plus petit**.
- Trie selon la **taille du fichier** (`x[1]`), en **ordre décroissant**.

Made with Gamma

Script_2.py

Explication 2 :

Fonction filter_function() (Filtrage des fichiers)

- Filtre les fichiers en fonction :
  - d'une **taille minimale** en Mo.
  - d'un **nombre maximum** de fichiers à conserver.
- Convertit les **Mo en octets** (1 Mo = 1 048 576 octets).
- Ne garde que les fichiers **plus grands** que la taille minimale.
- Limite le **nombre de fichiers retournés** à NB_MAXI_FICHIERS.
- Convertit la liste des fichiers en **fichier JSON**.
- Remplace \ par \\ (évite les erreurs d' échappement dans JSON).
- **Crée et enregistre** le fichier JSON avec une indentation **lisible**.
- Affiche un message une fois l'export terminé.

Fonction main()

- Fonction principale du script.
- Vérifie si un **argument** (répertoire) est passé, sinon utilise le répertoire **courant**.
- Affiche le **répertoire analysé**.
- Explore le dossier et affiche le **nombre de fichiers trouvés**.
- Trie les fichiers du plus gros au plus petit.
- Définit les **critères de filtrage**.
- Filtre et affiche le nombre de fichiers conservés.
- Enregistre les fichiers dans un fichier gros_fichiers.json.

Lancement du script

- Exécute main() si le script est lancé directement.

Résumé 💡

- **Explore** un dossier et récupère les fichiers avec leur taille.
- **Trie** les fichiers du plus gros au plus petit.
- **Filtre** selon une taille minimale et un nombre max.
- **Enregistre** les résultats dans un fichier JSON.

Made with Gamma

Script_3.py

Programme :

```
import json import sys import random from pathlib import Path from PyQt5.QtWidgets import QApplication from PyQt5.QtGui import QColor from Creation_Onglets import Onglets from Creation_Camembert import Camembert from Creation_Legendes import Legendes from Creation_Boutons import Boutons import subprocess

NB_MAXI_FICHIERS = 100 NB_LEGENDES_PAR_PAGE = 25 JSON_FILE = "gros_fichiers.json"

def charger_donnees_json(fichier_json): """Charge les données du fichier JSON et vérifie sa conformité.""" with open(fichier_json, "r") as f: data = json.load(f) if not all( isinstance(item, list) and len(item) == 2 and isinstance(item[0], str) and isinstance(item[1], int) for item in data): raise ValueError("Le fichier JSON ne respecte pas la structure attendue.") return data

def generer_couleurs(n): """Génère une liste de n couleurs aléatoires au format QColor.""" return [QColor(random.randint(0, 255), random.randint(0, 255), random.randint(0, 255)) for _ in range(n)]

def creer_script_suppression(liste_fichiers, etats_cases): """Génère un script PowerShell compatible multi-plateforme pour supprimer les fichiers sélectionnés.""" script_path = Path("suppression_fichiers.ps1") with script_path.open("w", encoding="utf-8") as f: f.write("Write-Output "Script de suppression de fichiers"\n") f.write("$reponse = Read-Host "Confirmer suppression (OUI)"\n") f.write("if ($reponse -eq 'OUI') {\n") f.write(" $confirmation = Read-Host "Etes-vous sûr ? (OUI)"\n") f.write(" if ($confirmation -eq 'OUI') {\n") for fichier, etat in zip(liste_fichiers, etats_cases): if etat: path_correct = Path(fichier[0]).as_posix() f.write(f' Remove-Item -Path "{path_correct}" -Force\n') f.write(" } else { Write-Output "Opération annulée" }\n") f.write("} else { Write-Output "Opération annulée" }\n") print("Script de suppression généré.")
```

```
# Exécuter le script PowerShell
try:
    subprocess.run(["powershell", "-File", str(script_path)], check=True)
except subprocess.CalledProcessError as e:
    print(f"Erreur lors de l'exécution du script PowerShell: {e}")
```

```
def main(): """Programme principal d'affichage graphique.""" app = QApplication(sys.argv)
fenetre = Onglets()
```

```
liste_fichiers = charger_donnees_json(JSON_FILE)
liste_couleurs = generer_couleurs(len(liste_fichiers))

fromage = Camembert(liste_fichiers, liste_couleurs)
layout_fromage = fromage.dessine_camembert()
fenetre.add_onglet("Camembert", layout_fromage)

listes_legendes = []
for num_page in range((len(liste_fichiers) + NB_LEGENDES_PAR_PAGE - 1) //
NB_LEGENDES_PAR_PAGE):
```

Made with Gamma

Script_3.py

Programme :

```
    # Pass the starting index and the number of legends per page to Legendes
    legende = Legendes(liste_fichiers, liste_couleurs, num_page *
NB_LEGENDES_PAR_PAGE, NB_LEGENDES_PAR_PAGE)
    listes_legendes.append(legende)
    layout_legende = legende.dessine_legendes()
    fenetre.add_onglet(f"Légende {num_page + 1}", layout_legende)


def callback_suppression():
    etats = [etat for legende in listes_legendes for etat in
legende.recupere_etats_cases_a_cocher()]
    creer_script_suppression(liste_fichiers, etats)

repertoire_base = Path.cwd()  # Assurez-vous que le répertoire de base est correctement
transmis
ihm = Boutons(str(repertoire_base), callback_suppression)
layout_ihm = ihm.dessine_boutons()
fenetre.add_onglet("IHM", layout_ihm)

fenetre.show()
sys.exit(app.exec_())
```

if **name** == "**main**": main()

Explication 1 :

Importation

- `json` : Permet de lire et écrire des fichiers JSON.
- `sys` : Permet d'interagir avec les arguments et la sortie du programme.
- `random` : Utilisé pour générer des couleurs aléatoires.
- `Path` de `pathlib` : Manipule les chemins de fichiers de manière portable.
- `PyQt5.QtWidgets` & `PyQt5.QtGui` : Importation des classes pour l' interface graphique.
- `subprocess` : Utilisé pour exécuter des commandes PowerShell.
- Importation de classes personnalisées (`Onglets`, `Camembert`, `Legendes`, `Boutons`) depuis d'autres fichiers.

Déclaration de constantes

- `NB_MAXI_FICHIERS` : Nombre maximal de fichiers affichés.
- `NB_LEGENDES_PAR_PAGE` : Nombre maximal de légendes affichées par onglet.
- `JSON_FILE` : Nom du fichier JSON contenant les fichiers détectés.

Fonction pour charger les données JSON

- Ouvre et charge le fichier JSON.
- Vérifie que chaque entrée est une liste `[chemin_fichier, taille]` avec :
- `chemin_fichier` : `str`
- `taille` : `int`
- Si le fichier JSON ne respecte pas la structure, lève une erreur.

Fonction pour générer des couleurs aléatoires

- Génère une liste de `n` couleurs aléatoires au format `QColor` (utilisé pour l'affichage graphique).

Made with Gamma

Script_3.py

Explication 2 :

Fonction pour créer un script PowerShell de suppression

- Génère un script PowerShell (`suppression_fichiers.ps1`) qui demande confirmation avant de supprimer les fichiers sélectionnés.
- `Read-Host` : Demande une entrée utilisateur pour confirmer l’ action.
- `Remove-Item -Path` : Commande PowerShell pour supprimer un fichier.

Exécution du script PowerShell

- `subprocess.run()` : Exécute le script PowerShell.
- Gestion des erreurs si l’ exécution échoue.

Fonction principale

- Crée une application PyQt5.
- `Onglets()` : Initialise une fenêtre avec un système d'onglets.

Chargement des données

- Charge les fichiers détectés depuis le JSON.
- Génère une liste de couleurs associées.

Création du camembert

- `Camembert(liste_fichiers, liste_couleurs)` : Crée un graphique circulaire (camembert) avec les fichiers détectés.
- `dessine_camembert()` : Génère le layout graphique du camembert.
- `fenetre.add_onglet("Camembert", layout_fromage)` : Ajoute un onglet contenant le graphique.

Création des légendes

- Crée plusieurs onglets pour afficher les légendes (max `NB_LEGENDES_PAR_PAGE` par page).

Création du bouton de suppression

- `callback_suppression()` : Récupère les cases cochées et génère le script de suppression.
- `Boutons(str(repertoire_base), callback_suppression)` : Crée une interface avec des boutons pour gérer la suppression.
- `dessine_boutons()` : Génère le layout.
- Ajoute un onglet "IHM" pour cette interface.

Lancement du programme

- `fenetre.show()` : Affiche la fenêtre principale.
- `sys.exit(app.exec_())` : Lance l'application et gère l'exécution.

Exécution conditionnelle

- Vérifie si le script est exécuté directement et non importé.
- Appelle `main()` pour démarrer l'application.

Résumé 💡

- Lit un fichier JSON contenant une liste de fichiers volumineux.
- Affiche ces fichiers sous forme de graphique et de liste.
- Permet de sélectionner des fichiers et de les supprimer via PowerShell.

Made with Gamma